Empowering Users with Algorithmic Control and Transparency in Tourism Recommender Systems: A DSA-Compliant Approach

DAVID GALLOB, Technical University of Munich, Germany

Modern recommender systems that users encounter in daily life are very often "black boxes," meaning that users rarely know how their recommendations are generated and have no control over which items are recommended. In tourism, this leads to a lack of trust and user satisfaction, as users can never be sure if their recommendations are truly optimal or influenced by the profit-maximization goals of the platform. The EU has therefore introduced the Digital Services Act (DSA), a framework that requires platforms to explain their algorithms in plain, understandable language and offer users a profiling-free alternative. In this paper, a web-based tourism recommender system prototype is presented that addresses these issues by implementing two algorithms. The application offers detailed explanations of the underlying decision algorithms and shows, for each result, how the recommendation was computed. The two algorithms implemented are Weighted Multi-Attribute Utility Theory (MAUT), which lets users set weights on the different attributes (e.g., star rating, distance to the center, price) on a scale from 0 to 10, and Constraint-Based Filtering, where the user sets hard limits on the attributes (e.g., price must be below 100 USD per night). This paper details the system architecture, the mathematical foundations of the implemented algorithms, and the UI features designed to explain these algorithms to the user.

Additional Key Words and Phrases: Recommender Systems, User Control, Multi-Attribute Utility Theory, Constraint Filtering, Digital Services Act, Explainable AI, E-Tourism

1 INTRODUCTION

In day-to-day life, recommender systems are ubiquitous, ranging from online shopping and video streaming to hotel and flight recommendation platforms [1]. Many prominent recommender systems (RS) do not let users know how their recommendations are generated and offer no direct means of influencing the outcome [2]. These systems are often built on complex algorithms (mainly neural networks) that process enormous amounts of user data to generate suggestions. They rarely explain their underlying logic to users. This lack of transparency can hinder trust and user satisfaction, leading to frustration and cognitive overhead. The tourism domain, where financial expenditure and emotional investment are generally high, is particularly affected by this issue [8].

The field of Human-Computer Interaction (HCI) has focused research on user influence and transparency (explainability) to address these shortcomings [6, 12]. Instead of using automated profiling, interactive recommender systems give users the ability to influence the recommendation process by adjusting sliders representing attribute weights or setting hard constraints on the attributes (e.g., price or distance).

The demand for transparency is further fueled by the European Union's Digital Services Act (DSA), which governs online platforms and aims to protect users' rights. The DSA mandates that online platforms must provide users with transparent information about the main parameters of their recommender systems in plain, understandable language and offer options to influence them (Article 27) [3]. Article 38 further obliges very large online platforms (VLOPs) to provide users with at least one option for their recommender systems that is not based on profiling. These articles require developers to shift from black-box profiling towards transparent and user-controlled platforms.

In this paper, the design decisions, architecture, and implementation of a web-based tourism recommender system prototype that is compliant with the DSA are detailed. It addresses both transparency and user-influence goals by implementing two algorithms:

Author's address: David Gallob, david.gallob@tum.de, Technical University of Munich, School of Computation, Information and Technology, Munich, Germany.

2026. ACM 2476-1249/2026/6-ART1
https://doi.org/

(1) **Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT)**: This algorithm uses sliders to determine weights for hotel attributes (price, distance to center, star rating, guest review score, and public transport access). To prevent user confusion regarding percentages summing up to 100%, the importance weights are defined on a 0 to 10 scale (where 0 represents completely unimportant and 10 represents the most important). The algorithm normalizes the attributes and computes a weighted utility score for each hotel. Finally, the hotels are ranked according to their utility score (percentage match).
(2) **Algorithm B: Constraint-Based Filtering**: In Constraint-Based Filtering, the user defines hard limits for each attribute (e.g., maximum price or minimum rating). All hotels that do not meet the constraints are filtered out. This allows users to specify their preferences in a clear and understandable way without the need for complex mathematical calculations.

The prototype features a dynamic strategy selection landing wizard that ensures transparency panels are not shown prematurely. Once a strategy is chosen, a dynamic banner is displayed that explains the chosen algorithm in plain language and offers a button that, when pressed, shows the mathematical formula of the chosen algorithm for enhanced transparency. The calculated results also offer a collapsible mathematical breakdown (displayed only upon user intent to avoid overwhelming uninterested users) showing a step-by-step calculation of how the hotel's score was computed. This allows users to understand how the results are generated and how their inputs influence the outcome. The calculation breakdown is generated using KaTeX and renders the exact mathematical formulas and normalization equations used in the calculations, allowing users to trace how their inputs directly translate into the presented results.

1.1 Paper Structure

The remainder of this paper is structured as follows: In section 2, related work in current tourism recommender systems, MAUT, and explanation interfaces is presented. section 3 details the system's architecture and the mock dataset. section 4 describes the mathematical foundations of the two implemented algorithms. section 5 details the UI design and transparency features. section 6 proposes a protocol for a future user study and extensions, and section 7 concludes the paper. The Appendix contains the complete hotel database, author information and contribution details, and Generative AI disclosures.

2 RELATED WORK

Unlike recommendations for low-cost consumer goods like movies and music, the development of recommender systems in the tourism domain faces unique challenges [2]. In tourism, decisions require significant rational resources, are infrequent, and involve multiple decision attributes. In contrast to low-cost consumer goods, vacation planning involves high financial risk and high emotional expectations, leading to a significant time commitment. Users therefore gather extensive information and compare multiple options, and they need to express specific constraints before making a final decision [8].

Traditional recommender systems in the tourism domain are primarily categorized into three classes [4, 9]:

- **Collaborative filtering:** Builds on the historical preferences of similar users but suffers from the cold-start problem. Specifically, new hotels without reviews and infrequent travelers without historical ratings lead to poor recommendations.
- **Content-based:** These systems match item features with the preference profiles of the user. While they can perform well, they are often unable to explain their recommendations or adapt to rapid changes in user needs (e.g., business travelers seeking central locations versus families seeking budget accommodations).
- **Hybrid:** These recommender systems combine multiple approaches.

To address these limitations, multi-attribute frameworks, such as Multi-Attribute Utility Theory (MAUT) for decision-making, have increasingly been used in interactive recommender systems [10]. MAUT is a mathematical

framework that enables the evaluation of alternatives with potentially conflicting attributes [5]. The algorithm defines a utility function for every attribute (e.g., mapping location to a utility between 0 and 1) and calculates a weighted sum. MAUT, as shown by Schmitt et al. in 2002 [10], is particularly useful in interactive recommender systems, as it takes human decision-making processes into account (i.e., user-defined weights). It allows for deficiencies (e.g., a high price) to be compensated for by a high utility in another attribute (e.g., an excellent location).

Constraint-based recommender systems are, in contrast, non-compensatory. They do not calculate a score, but filter out items that violate user-defined limits based on explicit knowledge about the items [8]. This approach is easy to understand and does not require user profiling, perfectly aligning with the EU's DSA [3].

Earlier research has shown that the success of interactive recommender systems heavily relies on their UIs [7, 11]. Therefore, providing explanations in the UI improves user understanding, trust, and perceived control over the system [12]. Knijnenburg et al. [6] showed that their proposed framework, which offers user control paired with high explainability, leads to higher user satisfaction and acceptance. The integration of sliders for defining weights, mathematical breakdowns, and transparency panels investigated in this paper builds on these findings to construct an explainable and transparent UI that enhances user satisfaction and meets the DSA requirements.

3 SYSTEM ARCHITECTURE AND IMPLEMENTATION

The prototype is designed as a web application. The technical architecture consists of React, Vite for the build system, and vanilla CSS for layout and styling. Client-side math rendering is powered by KaTeX, a fast LaTeX math rendering library, integrated via a custom React wrapper. The system architecture is structured to support real-time user interaction.

The application state manages:

- The selected city (Munich or Vienna).
- The selected recommendation algorithm (Weighted MAUT or Constraint-Based Filtering).
- The five attribute weights ($w_i \in [0, 10]$) for the Weighted MAUT mode.
- The five attribute thresholds for the Constraint-Based Filtering mode.
- The global formula toggle, which controls the display of the explanation board and mathematical formulas.

The dataset, stored in a static module (`src/data.js`), contains 20 hotels (10 for each city). These accommodations represent a realistic and diverse range, spanning from low-budget hostels to luxury five-star hotels. Each hotel is uniquely defined by an identifier, name, city, image, description, and five attributes:

(1) **Price per Night**: Ranging from $35 (Hostel Vienna West) to $550 (Mandarin Oriental Munich).
(2) **Distance to Center**: Distance from the historical city center (Stephansplatz in Vienna, Marienplatz in Munich), ranging from 0.1 km (Stephansplatz) to 28.0 km (Airport Express Inn Munich).
(3) **Star Rating**: Ranging from 1 (basic tourist class) to 5 stars (luxury class).
(4) **Guest Review Score**: A continuous rating between 1.0 (worst) and 10.0 (excellent), representing aggregated guest satisfaction.
(5) **Public Transport Access Score**: A rating between 1.0 (poor) and 10.0 (excellent), representing proximity to subway stations, tram lines, and frequency of transit services.

The application calculates and updates the recommendations in real-time as the user modifies the sliders or switches the algorithm. This renders a "calculate" button unnecessary, improving user friendliness. This is made possible by performing the calculations on the fixed dataset entirely client-side.

4 ALGORITHMIC FRAMEWORK

To support the user-controlled interaction model, this section explains the mathematical models behind the algorithms in the recommendation engine.

4.1 Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT)

This algorithm evaluates each hotel in the chosen city by mapping the hotel's attribute values to normalized utility scores and computing a weighted average. The process is performed in three steps:

4.1.1 Normalization of Attributes. As the attributes use incomparable units (e.g., USD, kilometers, stars), they must first be normalized to a standard scale $u(v) \in [0, 1]$, where 0 represents the worst possible utility and 1 represents the best possible utility.

The hotel attributes are split into cost metrics (where lower values are better, like price and distance) and benefit metrics (where higher values are better, like star ratings, guest review scores, and public transport access). The prototype implements two normalization strategies:

Relative Normalization: Used for cost metrics, as these values do not have clear upper and lower bounds. Therefore, the items are compared to one another to define the thresholds. The normalized utility score $u(v)$ for an attribute value v of an item is calculated as:

$$u(v) = \frac{v_{\max} - v}{v_{\max} - v_{\min}} \tag{1}$$

where:

- v is the value of the hotel.
- $v_{\min}$ and $v_{\max}$ are the minimum and maximum values of this attribute of all hotels in the selected city.

Absolute Normalization: This is used for benefit attributes (stars, rating, and public transport access) because these values have clear intrinsic bounds for the best and worst options (e.g., stars or guest review ratings). In addition, normalizing these scores relatively could artificially exaggerate small differences (e.g., mapping a hotel with a guest rating of 8.5 to a utility of 0 simply because it is the lowest score in that city).

For stars (on a 1 to 5 scale), the utility is calculated as follows:

$$u_{\text{stars}}(v) = \frac{v - 1}{4} \tag{2}$$

where v is the number of stars of the hotel (from 1 to 5).

For the guest review rating scores and the public transport access score (both ranging from 1 to 10), the utility is calculated as follows:

$$u_{\text{rating/transport}}(v) = \frac{v - 1}{9} \tag{3}$$

where v is the guest review score or public transport access score of the hotel (from 1 to 10).

4.1.2 Weighted Aggregation. After the initial normalized attribute scores are calculated, the utility scores are weighted and summed. Let w_i be the user-defined weight for attribute i (where $w_i \in [0, 10]$), and u_i be the previously calculated normalized utility score of attribute i for the hotel. The total utility U is calculated as:

$$U = \frac{\sum_{i=1}^{5} w_i \cdot u_i}{\sum_{i=1}^{5} w_i} \times 100\% \tag{4}$$

The calculated hotels are sorted and shown to the user in descending order.

4.2 Algorithm B: Constraint-Based Filtering

The Constraint-Based Filtering algorithm does not calculate a mathematical match score or rank the hotels. Instead, it checks if the set of hotels in the selected city complies with the user-defined constraints. A hotel is in the result set (i.e., shown to the user) if and only if it satisfies the logical conjunction:

$$\text{Price} \leq P_{\max} \ \wedge \ \text{Distance} \leq D_{\max} \ \wedge \ \text{Stars} \geq S_{\min} \ \wedge \ \text{Rating} \geq R_{\min} \ \wedge \ \text{Transport} \geq T_{\min} \tag{5}$$

where:

- $P_{\max}$ is the maximum price threshold defined by the user.
- $D_{\max}$ is the maximum distance threshold defined by the user.
- $S_{\min}, R_{\min}, T_{\min}$ are the minimum stars, guest review rating, and public transport access scores thresholds defined by the user.

This filtering logic is non-compensatory — hotels are not compared to each other or ranked relatively. All hotels are simultaneously checked and excluded from the results if a single constraint is violated.

4.3 Comparison of Decision Logic

The two models are fundamentally different in their decision-making styles. The primary difference is that Weighted MAUT, due to its ranking nature, compiles a complete, ordered list of all hotels, meaning that even lower-matching options are displayed. In contrast, Constraint-Based Filtering, which is non-compensatory, excludes non-matching hotels to deliver a focused result set, though it does not indicate how well each remaining hotel satisfies the user's preferences. The differences are summarized in Table 1.

Table 1. Comparison of the Implemented Algorithmic Paradigms

Dimension	Algorithm A: Weighted MAUT	Algorithm B: Constraint Filtering
Decision Logic	Compensatory (trade-offs allowed)	Non-compensatory (hard boundaries)
Output Type	Ranked list with continuous match scores	Unordered subset of matching items
User Controls	Relative importance weights ($w_i \in [0, 10]$)	Threshold limits ($P_{\max}, D_{\max}$, etc.)
Cognitive Style	Optimizing	Screening / Satisficing
DSA Compliance	Explains weights	Explains inclusion/exclusion criteria
Profiling Dependency	None (Profiling-free, real-time input)	None (Profiling-free, real-time input)

4.4 Quality Assurance and Algorithmic Verification

To ensure that the prototype is stable and mathematically correct, the recommendation engine was implemented in a standalone module (`src/recommenderEngine.js`). A unit testing suite (`run-tests.js`) was created to verify the module, consisting of exactly 60 test cases (30 for Weighted MAUT and 30 for Constraint-Based Filtering) that test edge cases and various input parameter patterns.

4.4.1 MAUT Logic Verification (30 Cases). The 30 MAUT test cases evaluate the logic of normalization and scoring:

- **Weight Boundary Conditions (5 cases)**: These tests verify that equal weights result in identical rankings, all-zero weights do not cause a division-by-zero error, and input weights are strictly bounded within $[0, 10]$.
- **Price Relative Cost Normalization (5 cases)**: Confirms that Relative Normalization works as expected by mapping the lowest price to a utility of 1.0, the highest price to a utility of 0.0, and intermediate prices proportionally. They also verify that no division by zero occurs when all items have the same price.

- **Distance Relative Cost Normalization (5 cases)**: Verifies that the closest hotel maps to a utility of 1.0, the furthest hotel to a utility of 0.0, and out-of-bound distances are clamped.
- **Absolute Normalizations (5 cases)**: Asserts that absolute utility mappings for stars (1-to-5 scale) and guest ratings/public transport scores (1-to-10 scale) are correct.
- **Weight Dominance and Flipped Rankings (5 cases)**: These cases verify that highly weighted attributes dominate the ranking and that adjusting weights dynamically updates the ranking order.
- **Robustness and Sorting (5 cases)**: This suite verifies edge cases such as empty list inputs, null inputs, missing weight keys, strict descending sorting order of match scores, and correct formatting of breakdown scores.

4.4.2 Constraint-Based Filtering Verification (30 Cases). The 30 filtering test cases check the correctness of the logical conjunction:

- **Individual Attribute Constraints (25 cases)**: These tests check the price, distance, star rating, guest review scores, and public transport access scores separately (5 tests per attribute). They verify inclusion and exclusion boundaries, including scenarios where constraints yield an empty set or include all items, and check that missing values fallback to default thresholds.
- **Conjunction and Robustness (5 cases)**: These tests verify that combined constraints filter conjunctively, that high scores in one attribute cannot compensate for violations in another (non-compensatory property), and that empty/null datasets are handled gracefully.

All test cases run against the test suite successfully and pass.

5 USER INTERFACE DESIGN AND EXPLAINABILITY

The prototype's frontend interface was developed with the assistance of Gemini 3.5[1] in a modern style with smooth animations to create a premium, modern experience. The UI was designed to feature maximum explainability for users.

5.1 Landing Page for Algorithm Selection

Prior iterations showed detailed algorithm explanations before selection, leading to overwhelming new users (see the previous cluttered layout in Figure 1). This issue was tackled by implementing a landing page that explains the available algorithms in plain language and prompts the user to select an algorithm. The user is shown two cards to select from:

- **Weighted MAUT Card**: Focuses on compensatory trade-offs. It explains that the user can rate the importance of attributes on a scale of 0 to 10, resulting in a ranked list of hotels with percentage match scores.
- **Constraint-Guided Card**: Focuses on strict non-compensatory boundaries. It explains that the user can set threshold limits to eliminate hotels violating any of the criteria.

Afterwards, the user may set the weights or constraints depending on their chosen algorithm and is shown a transparency board that details the chosen algorithm. This landing-page approach makes the prototype more transparent, as it explains the logic to all users from the outset. The user may always return to this screen and change their choice by clicking on a "Switch Algorithm" button or change the algorithm directly using the controls on the main page.

[1] https://deepmind.google/models/gemini/flash/

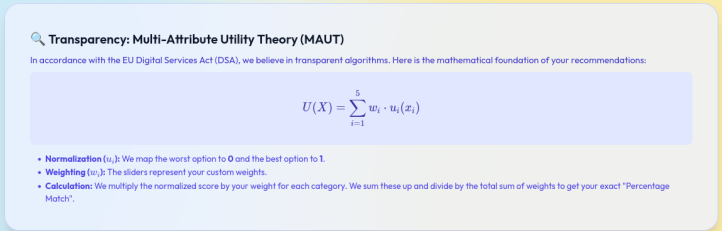

Fig. 1. Previous cluttered transparency board design that led to premature disclosure of complex information.

5.2 Global Explanation Banners

At the top of the main page, there is a transparency board that changes color and content based on the selected algorithm:

- In the **Weighted MAUT** mode, the banner is blue and explains the logic of the Weighted MAUT algorithm and the sliders (see Figure 2).
- In the **Constraint-Guided** mode, the banner is green, explaining the logic behind the non-compensatory filtering approach (see Figure 3).

Both banners use simple language to avoid overwhelming users with mathematical details. For interested users, the banners include a "Show Mathematical Details" button. When clicked, the banner renders the underlying formulas in LaTeX using KaTeX (Equation 4 for MAUT, and Equation 5 for Constraint-Guided Filtering) and the panel displays an explanation of the variables.

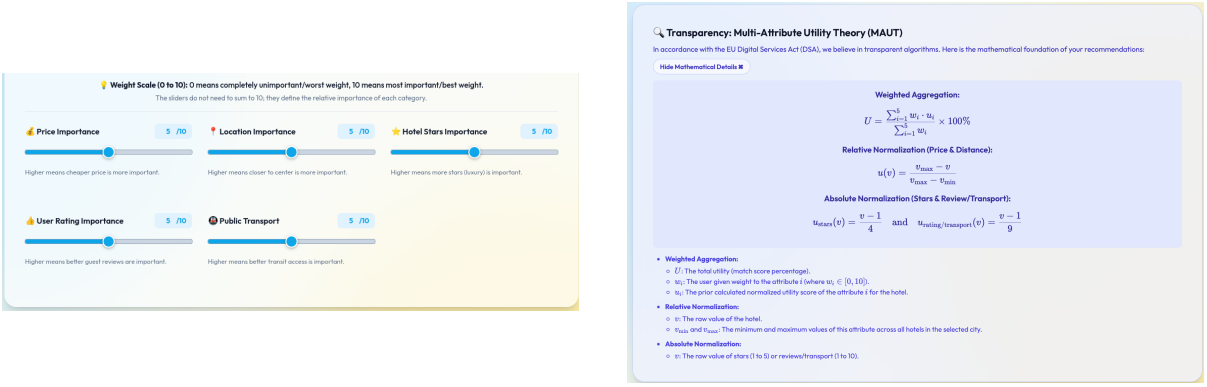

Fig. 2. Weighted MAUT user controls (left) and the dynamic blue explanation banner showing algorithmic details (right).

5.3 On-Demand Collapsible Math Breakdowns

In Weighted MAUT mode, after the user has set the sliders, the user can inspect the calculated results. Each hotel card offers a "Show Mathematical Breakdown" button to explain how this particular score (match percentage) was calculated (see Figure 4).

This feature ensures that the prototype offers absolute transparency while avoiding cognitive overload for uninterested users. The user can understand why a hotel is ranked in its current position and can compare it to

Fig. 3. Constraint-based filtering interface featuring threshold sliders and the dynamic green explanation banner.

higher- or lower-ranked hotels. By adjusting the sliders, the user can also check the mathematical impact of the slider weights on the results. This eliminates the perception of the recommender system as a black box.

Fig. 4. Collapsible step-by-step mathematical breakdown showing the exact utility calculations for a hotel.

6 FUTURE WORK

While the presented prototype offers many transparency features and demonstrates their feasibility in the tourism domain, research and further development are still required.

6.1 The Hybrid Algorithmic Approach

The approaches used in this prototype have clear strengths and weaknesses:

- The Weighted MAUT algorithm compares hotels with one another but does not filter out irrelevant items. For instance, a user interested only in low-budget hotels will still be recommended a high-priced hotel (albeit with a low match percentage).
- The Constraint-Based mode filters out these irrelevant options but does not rank the remaining hotels, leading to higher user involvement as they must find the best-matching hotel themselves.

Therefore, a hybrid approach, which combines their respective strengths and eliminates their weaknesses, is proposed for future work:

(1) **Stage 1: Constraint Filtering (Non-Compensatory screening)**: The system applies Constraint-Based Filtering first to filter out hotels that do not meet the user's basic requirements (e.g., price > $250 or distance > 10 km). This ensures that the candidates for further processing are limited to feasible options.
(2) **Stage 2: Weighted MAUT (Compensatory ranking)**: The system applies the user's preference weights to rank the preprocessed hotel candidates. This ensures that users only see hotels relevant to their needs (e.g., the aforementioned low-budget traveler will not see a luxury five-star hotel with a low match score of 8%).

This hybrid approach maintains full transparency, remains profiling-free, and reduces cognitive load. It represents a rational approach for human decision-making where unacceptable alternatives are filtered out first, and the best-fitting alternative is then identified from the remaining options.

6.2 User Study

To evaluate the effectiveness of the applied transparency features in this prototype, a user study is proposed for future work. A fitting experimental design would be a within-subjects study with $N = 30$ participants, where each participant performs travel planning tasks in each of the following three recommender system environments:

(1) **Environment 1 (Traditional Black Box)**: A recommender system that displays hotel cards with the ranking logic completely hidden from the user.
(2) **Environment 2 (Weighted MAUT with On-Demand Explanations)**: A recommender system that uses the slider-based UI with collapsible math breakdown panels.
(3) **Environment 3 (Constraint Filtering with Explanations)**: A constraint-based filtering recommender system using thresholds.

The ordering of conditions will be counterbalanced, and participants will answer a questionnaire after experiencing each recommender system environment to track their impressions immediately and avoid confusion between the different systems. The results of this study will aid in enhancing the UI, helping to strike a balance between ease of use and mathematical transparency.

7 CONCLUSION

The DSA and modern user-experience expectations pose challenges for traditional black-box recommender systems. In this work, a transparent yet user-friendly recommender system prototype was implemented to comply with the EU's DSA. The prototype offers two distinct algorithms: a compensatory Multi-Attribute Utility Theory (MAUT) algorithm, which uses user-defined weights to rank hotels, and a non-compensatory Constraint-Based Filtering algorithm, which checks hotels against user-defined thresholds and excludes those that violate them. The prototype implements transparency features that explain the logic behind the algorithms step-by-step and, for the Weighted MAUT algorithm, detail how each hotel's score is computed. These features allow interested users or regulators to dive deep into the mathematical foundations of the results or the recommendation engine itself. This prototype architecture serves as a blueprint for developers seeking to combine modern design, intuitive user controls, and complete transparency in compliance with the DSA in recommender system design.

REFERENCES

[1] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering* 17, 6 (2005), 734–749. https://doi.org/10.1109/TKDE.2005.99

[2] Joan Borras, Antonio Moreno, and Aida Valls. 2014. Intelligent tourism recommender systems: A survey. *Expert Systems with Applications* 41, 16 (2014), 7370–7389. https://doi.org/10.1016/j.eswa.2014.06.007

[3] European Parliament and Council of the European Union. 2022. Regulation (EU) 2022/2065 of the European Parliament and of the Council of 19 October 2022 on a Single Market For Digital Services and amending Directive 2000/31/EC (Digital Services Act). http://data.europa.eu/eli/reg/2022/2065/oj.
[4] Mohamed Elyes Ben Haj Kbaier, Hela Masri, and Saoussen Krichen. 2017. A Personalized Hybrid Tourism Recommender System. In *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*. 244–250. https://doi.org/10.1109/AICCSA.2017.12
[5] Ralph L. Keeney and Howard Raiffa. 1993. *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press.
[6] Bart P. Knijnenburg, M. C. Willemsen, Z. Gantner, H. Soncu, and C. Newell. 2012. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction* 22, 4-5 (2012), 441–504. https://doi.org/10.1007/s11257-011-9118-4
[7] Sean M. McNee, John Riedl, and Joseph A. Konstan. 2006. Being accurate is not enough: How accuracy metrics have hurt recommender systems. In *CHI '06 Extended Abstracts on Human Factors in Computing Systems*. 1097–1101. https://doi.org/10.1145/1125451.1125659
[8] Francesco Ricci. 2011. Travel recommender systems. In *Recommender Systems Handbook*. Springer, 527–556. https://doi.org/10.1007/978-0-387-85820-3_17
[9] Joy Lal Sarkar, Abhishek Majumder, Chhabi Rani Panigrahi, Sudipta Roy, and Bibudhendu Pati. 2023. Tourism recommendation system: a survey and future research directions. *Multimedia Tools and Applications* 82, 6 (2023), 8983–9027. https://doi.org/10.1007/s11042-022-12167-w
[10] Christian Schmitt, Dietmar Dengler, Michael Bauer, et al. 2002. The MAUT-Machine: An Adaptive Recommender System. In *Proceedings of the ABIS Workshop* (Hannover, Germany).
[11] Kirsten Swearingen and Rashmi Sinha. 2001. Beyond Algorithms: An HCI Perspective on Recommender Systems. In *Proceedings of the SIGIR 2001 Workshop on Recommender Systems*.
[12] Nava Tintarev and Judith Masthoff. 2007. A Survey of Explanations in Recommender Systems. In *Proceedings of the 23rd International Conference on Data Engineering Workshop (ICDEW)*. 801–810. https://doi.org/10.1109/ICDEW.2007.4401070

A COMPLETE HOTEL DATABASE

Table 2 presents the complete dataset of 20 hotels used in the recommender system prototype, split between Vienna (v1–v10) and Munich (m1–m10).

B AUTHOR INFORMATION AND CONTRIBUTION DETAILS

As a single-author project, I, David Gallob, worked entirely alone on this project. This involved all aspects of the lab course, including the development and testing of the prototype, the creation and delivery of all milestone and final presentations, and the writing of this report.

C GENERATIVE AI DISCLOSURE

This project utilized Generative AI tools (specifically Google DeepMind's Gemini 3.5 Flash) during development. The AI was employed as a pair-programming assistant to:

- Draft and optimize the user interface layouts and CSS styling rules.
- Debug and recommend JavaScript state handling best practices.
- Assist in structuring, polishing, and editing the LaTeX code of this scientific report.

All design decisions, algorithmic formulations, bug resolutions, and report text were audited, verified, and approved by the human author to ensure compliance with the scientific standards of the Technical University of Munich.

Table 2. Hotel Database Attributes

ID	Hotel Name	Price ($/night)	Distance (km)	Stars	Rating (/10)	Transport (/10)
		Vienna Hotels				
v1	Grand Imperial Vienna	350	0.5	5	9.6	9.5
v2	Boutique Hotel am Stephansplatz	180	0.1	4	9.2	10.0
v3	Danube Riverside Hotel	95	4.5	3	8.0	6.5
v4	Schönbrunn Palace Suites	290	6.0	5	9.8	8.0
v5	Hostel Vienna West	35	3.2	1	7.5	9.0
v6	Art Deco Hotel Prater	140	2.8	4	8.7	8.5
v7	Economy Inn Vienna	60	5.5	2	6.8	5.0
v8	Museum Quarter Retreat	210	1.2	4	9.4	9.0
v9	Green Oasis Hotel	110	7.5	3	8.9	4.0
v10	The Ritz-Carlton Vienna	450	0.8	5	9.7	10.0
		Munich Hotels				
m1	Bayerischer Hof	420	0.4	5	9.5	9.5
m2	Marienplatz Business Hotel	160	0.2	4	8.8	10.0
m3	Oktoberfest Lodge	85	2.5	2	7.2	8.0
m4	Englischer Garten Retreat	250	3.0	4	9.3	7.0
m5	Schwabing Youth Hostel	40	4.0	1	8.1	8.5
m6	Olympic Park Hotel	130	6.5	3	8.5	7.5
m7	Airport Express Inn	90	28.0	3	7.8	9.0
m8	Viktualienmarkt Boutique	190	0.6	4	9.1	9.5
m9	Isar River View	210	1.8	4	8.9	6.0
m10	Mandarin Oriental Munich	550	0.5	5	9.8	9.5